

ADR 004 — Diseño Genérico de los TDA e Interfaces Compartidas

Estado

Aceptado.

Contexto

El sistema utiliza múltiples Tipos de Datos Abstractos para resolver distintos problemas de almacenamiento, búsqueda y procesamiento de información.

Durante el diseño inicial se observó que varias estructuras comparten operaciones fundamentales como inserción, eliminación, consulta de tamaño y verificación de contenido.

Mantener implementaciones completamente independientes generaría duplicación de código y dificultaría la estandarización de comportamientos.

Decisión

Se decidió definir una interfaz común denominada:

Contenedor<T>

La misma establece las operaciones básicas que deberán implementar las estructuras de datos utilizadas dentro del sistema.

Las operaciones definidas son:

```
insertar(elemento)
eliminar(clave)
estaVacio()
tamano()
```

Implementación Genérica

Se decidió implementar los TDA utilizando tipos genéricos.

De esta forma las estructuras podrán reutilizarse con distintos tipos de datos sin necesidad de crear implementaciones específicas para cada entidad del sistema.

Ejemplos:

```
AVL<Producto>
```

```
Queue<Pedido>
```

```
Stack<Movimiento>
```

```
MinHeap<Producto>
```

```
Lista<Ruta>
```

Unificación de Comportamientos

La interfaz común permite que todas las estructuras compartan una base conceptual uniforme.

Aunque cada TDA implemente internamente algoritmos diferentes, la interacción externa mantiene una lógica consistente para el desarrollador.

Esto facilita:

- Aprendizaje del código.
- Reutilización de componentes.
- Integración entre módulos.
- Mantenimiento futuro.

Extensibilidad

La utilización de una interfaz compartida permite incorporar nuevas estructuras sin modificar el resto de la arquitectura.

Toda nueva implementación deberá respetar el contrato definido por la interfaz Contenedor.

Ejemplos futuros:

```
HashTable<T>
```

```
BinarySearchTree<T>
```

```
PriorityQueue<T>
```

Independencia de la Lógica de Negocio

Los servicios del sistema interactúan con abstracciones y no con implementaciones concretas.

Esto reduce el acoplamiento entre la lógica de negocio y las estructuras utilizadas internamente.

Como consecuencia, una estructura puede reemplazarse por otra sin afectar significativamente el resto de la aplicación.

Organización del Proyecto

Las interfaces y estructuras de datos se mantendrán separadas de las entidades del dominio.

La organización general será:

```
interfaces/  
  Contenedor  
  
tda/  
  AVL  
  Stack  
  Queue  
  Lista  
  MinHeap  
  Grafo  
  
models/  
  Producto  
  Pedido  
  Ruta  
  UbicacionFisica  
  ItemPedido  
  
services/  
  Servicios de negocio
```

Esta separación facilita la identificación de responsabilidades dentro del proyecto.

Consecuencias

Positivas

- Reducción de duplicación de código.

- Mayor reutilización de componentes.
 - Consistencia entre las distintas estructuras.
 - Menor acoplamiento entre módulos.
 - Facilidad para incorporar nuevos TDA.
 - Mejor mantenibilidad del proyecto.
-

Negativas

- Mayor esfuerzo de diseño inicial.
 - Necesidad de respetar contratos definidos por las interfaces.
 - Posible incorporación de métodos no utilizados por algunas implementaciones específicas.
-

Decisiones Relacionadas

- Implementación manual de los TDA requeridos por la materia.
- Utilización de programación orientada a objetos.
- Modelado de entidades de dominio independientes de las estructuras de almacenamiento.
- Uso de tipos genéricos para maximizar reutilización y flexibilidad.